Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине: «Интеллектуальный анализ данных»

Тема: «PCA»

Выполнил:
Студент 4 курса
Группы ИИ-26
Яшков П. А.

Проверил:
Никитин С. К.

Брест 2026

Цель работы:

Научиться применять метод главных компонент PCA для уменьшения размерности и визуализации многомерных данных.

Общее задание

1. На выборке Seeds выполнить PCA двумя независимыми способами: вручную через numpy.linalg.eig и посредством sklearn.decomposition.PCA, для 2 и 3 компонент.
2. Построить 2D и 3D графики с различными цветами для трёх классов.
3. Вычислить потери информативности по собственным значениям и проанализировать результат.
4. Подготовить отчёт и исходный код для GitHub.

Сведения об эксперименте

Автор: Яшков П. А., группа ИИ-26, 4 курс. Вариант: 11. Все расчёты воспроизводимы: random_state/seed = 42. Исходники размещены в папке src/, результаты — src/results/.

Подготовка исходных данных и алгоритм

Данные Seeds: 210 объектов, 7 вещественных признаков, 3 сорта пшеницы. Класс хранится в последнем столбце и удаляется из матрицы признаков до PCA. Пропуски при наличии замещаются медианами. Признаки стандартизируются StandardScaler.

В ручной реализации построена ковариационная матрица стандартизованных признаков, вычислены собственные значения и векторы через numpy.linalg.eig. Собственные пары отсортированы по убыванию значений; данные спроецированы на 2 и 3 главные компоненты. Независимо построено решение sklearn.decomposition.PCA. Выполнена сверка собственных значений обоих методов.

Потери информативности вычислены как $L(k) = (1 - (\lambda 1 + ... + \lambda k) / (\lambda 1 + ... + \lambda n)) \times 100\ \%$.

Результат программы

Проекция	Сохранено	Потери	Разность eigenvalue
2D	88.98%	11.02%	1.42e-14
3D	98.67%	1.33%	1.42e-14

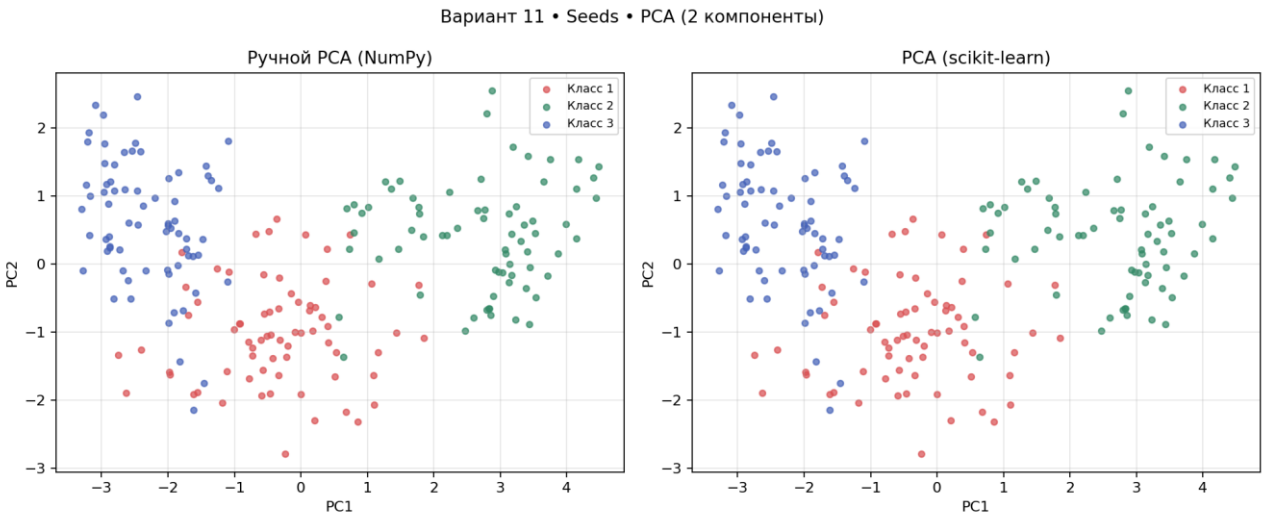


Рисунок 1 — Визуализация 2D: ручной PCA и scikit-learn.

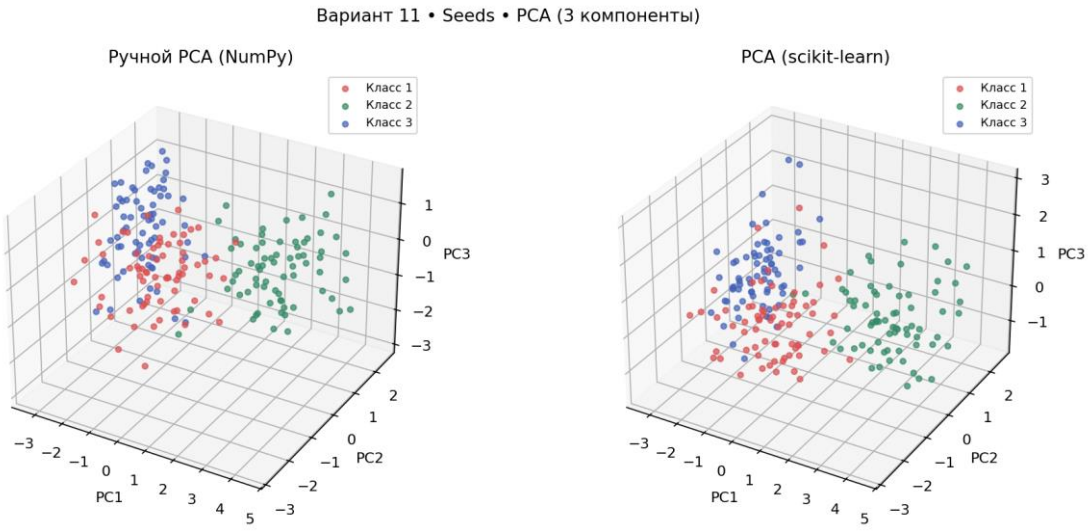


Рисунок 2 — Визуализация 3D: ручной PCA и scikit-learn.

Выводы о проделанной работе

1. Ручной PCA и sklearn дали согласованные собственные значения (отклонение порядка 1.4e-14). Различие направлений осей на рисунках может быть связано с произвольным знаком собственных векторов.
2. При переходе к двум компонентам сохраняется 88.98% общей дисперсии, теряется 11.02%.
3. Три компоненты сохраняют 98.67% дисперсии, потери составляют лишь 1.33%.
4. Разные классы Seeds в проекциях в целом образуют различимые области, хотя некоторые точки перекрываются; PCA не является классификатором и не гарантирует полного разделения сортов.

Приложение — код программы

Файл ИАД_1.py: полный исполняемый код размещён также в папке src/.

```
"""ИАД, лабораторная №1, вариант 11, Яшков П. А. | Seeds + PCA."""
from pathlib import Path
import json
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
ROOT=Path(__file__).resolve().parent
OUT=ROOT/'results';OUT.mkdir(exist_ok=True)
raw=pd.read_csv(ROOT/'data/seeds_dataset.txt',sep=r'\s+',header=None,na_values=['?'])
X=raw.iloc[:,:-1].apply(pd.to_numeric,errors='coerce');y=raw.iloc[:,-1].astype(int).values
X=X.fillna(X.median()).values
assert X.shape[1]==7 and len(X)==210
Z=StandardScaler().fit_transform(X)
cov=np.cov(Z,rowvar=False)
values,vectors=np.linalg.eig(cov)
order=np.argsort(values.real)[::-1];values=values.real[order];vectors=vectors.real[:,order]
results={'variant':11,'student':'Яшков П.
A.','rows':len(Z),'features':Z.shape[1],'classes':sorted(set(y.tolist())), 'methods':{}}
for k in (2,3):
 manual=Z@vectors[:,:k]
 sklearn_pca=PCA(n_components=k,svd_solver='full');sci=sklearn_pca.fit_transform(Z)
 # Собственные векторы могут отличаться знаком: сравниваем проекционные матрицы и объясненную дисперсию.
 explained=float(values[:k].sum()/values.sum());lost=100*(1-explained)
 diff=float(np.abs(sklearn_pca.explained_variance_ - values[:k]).max())
 assert diff<1e-8,(diff,k)

results['methods'][str(k)]={'retained_variance_percent':round(100*explained,4),'loss_percent':round(lost,4),'ei
genvalues_first':values[:k].round(6).tolist(),'max_variance_eigen_diff':diff}
 fig=plt.figure(figsize=(12.3,5.1))
 for idx,(arr,title) in enumerate([(manual,'Ручной PCA (NumPy)'),(sci,'PCA (scikit-learn)')],1):
  ax=fig.add_subplot(1,2,idx,projection='3d' if k==3 else None)
  for cls,color in zip(sorted(set(y)),['#da5153','#348a67','#4663b9']):
   a=arr[y==cls]
   if k==3:ax.scatter(a[:,0],a[:,1],a[:,2],s=14,alpha=.72,c=color,label=f'Класс {cls}')
   else:ax.scatter(a[:,0],a[:,1],s=17,alpha=.72,c=color,label=f'Класс {cls}')
  ax.set_title(title);ax.set_xlabel('PC1');ax.set_ylabel('PC2')
  if k==3:ax.set_zlabel('PC3')
  ax.legend(fontsize=8)
  if k==2:ax.grid(alpha=.3)
 fig.suptitle(f'Вариант 11 • Seeds • PCA ({k}
компоненты)');fig.tight_layout();fig.savefig(OUT/f'pca_{k}d.png',dpi=160);plt.close(fig)
 print(f'{k} компоненты: сохранено {explained*100:.2f}%, потеряно {lost:.2f}%')
(OUT/'metrics.json').write_text(json.dumps(results,indent=2,ensure_ascii=False),encoding='utf-8')
```